openssl-provider-vault: Transparent Cryptographic Key Isolation for OpenSSL 3 via HashiCorp Vault

Soren L. Hansen
sorenisanerd@gmail.com
Independent Researcher

Abstract

Private key exposure remains a primary cause of large-scale cryptographic failures. Hardware Security Modules mitigate this risk but impose high cost and operational complexity. Software-based secret-management systems such as HashiCorp Vault offer strong logical and network-level isolation guarantees at a fraction of the deployment friction—when deployed on a separate host, an adversary must compromise both the application process and the Vault cluster simultaneously to extract key material, a strictly stronger requirement than a single-system boundary—yet integrating them with applications that rely on standard TLS and signature APIs has historically required intrusive code changes.

We present *openssl-provider-vault*, an OpenSSL 3 provider that routes private-key operations such as signing and asymmetric encryption, together with HMAC operations, through Vault's transit secrets engine without any modification to applications or system libraries. The provider implements the standard OpenSSL 3 provider dispatch interface, making it a drop-in provider for the supported key-management, signing, asymmetric-cipher, MAC, and STORE operations. We describe the security architecture, the design trade-offs imposed by the OpenSSL 3 API, and lessons learned from bridging the impedance mismatch between a streaming cipher API and a prehash-oriented remote service.

CCS Concepts

• **Security and privacy** → **Key management**; *Cryptography*; • **Software and its engineering** → Software libraries and repositories.

Keywords

key management, OpenSSL provider, HashiCorp Vault, key isolation, transit encryption, cryptographic API

1 Introduction

The compromise of a private key is a catastrophic, often irreversible event. Once an adversary obtains the private key of a TLS certificate, a code-signing identity, or a long-lived authentication credential, they can silently impersonate the legitimate holder indefinitely. Notable incidents—from rogue certificate issuance to software supply-chain attacks—have involved leaked or compromised private keys as a central element.

The classical defense is an *Hardware Security Module* (HSM): a tamper-resistant device that holds the key and exposes only high-level operations (sign, decrypt). The private key never leaves the HSM boundary in cleartext. The drawback is cost—enterprise HSMs range from several hundred to several thousand dollars per unit—and operational complexity. Deployments must manage physical devices, driver stacks, firmware updates, and the coarse-grained concurrency model imposed by PKCS#11.

Software-defined secret-management systems, and HashiCorp Vault in particular, replicate the same conceptual boundary in a cloud-native package: a separate process (or cluster) holds keys, enforces access policy, and exposes only cryptographic operations through an authenticated HTTP API. Vault's *transit secrets engine* [1] functions as a "cryptography as a service" layer with built-in key versioning, automatic rotation, and a rich audit trail.

Integrating Vault into application-layer TLS or signing workflows is, however, non-trivial. Applications that use OpenSSL directly must either call Vault's HTTP API themselves—adding a heavyweight dependency and a significant refactoring burden—or depend on a thin wrapper library that re-implements enough of the EVP_PKEY interface to satisfy the caller. Neither approach is satisfactory at scale.

We resolve this tension by exploiting the *provider model* introduced in OpenSSL 3.0 [2]. A provider is a dynamically loaded module that implements one or more cryptographic operations through a well-defined dispatch table. Applications that use the standard `EVP_*` API are entirely unaware of the underlying provider; the runtime selects the appropriate one based on algorithm properties and the active library context. By packaging the Vault integration as a provider, we achieve full transparency: existing binaries, system TLS stacks, and OpenSSL-based

utilities obtain the isolation properties of Vault without any code changes beyond a configuration file entry.

The contributions of this paper are:

- A design and open-source implementation of `openssl-provider-vault`, an OpenSSL 3 provider backed by the Vault transit engine (§3, §4).
- An analysis of the security properties that the design provides, and the residual risks it does not eliminate (§6).
- A discussion of the impedance mismatches between OpenSSL's streaming cryptography model and Vault's prehash-oriented REST API, and how we resolve them (§5).

2 Background

2.1 The OpenSSL 3 Provider Model

OpenSSL 3 restructured its algorithm dispatch machinery around the concept of *providers* [2]. A provider is a shared library (or a statically compiled module) that exports a set of *operation implementations* through a C dispatch table. Each operation corresponds to an algorithm class—signature, asymmetric cipher, symmetric cipher, key management, MAC, digest, and so on—and is identified by a combination of algorithm name and property string.

The runtime selects a provider at algorithm instantiation time. If an application calls `EVP_DigestSignInit` with algorithm `"RSA-SHA256"` and the active library context has been configured with `openssl-provider-vault`, the provider's `signature` dispatch table is consulted first. From the application's perspective, the call is indistinguishable from the built-in software provider.

Key objects cross provider boundaries via a *key reference* mechanism. A provider exports keys as opaque byte sequences through `keymgmt_export` / `keymgmt_import`; a receiving provider reconstructs the local representation from those bytes. This decoupling lets openssl-provider-vault store only a lightweight handle to the remote key while still participating in the standard `EVP_PKEY` ownership model.

2.2 HashiCorp Vault Transit Engine

The Vault transit secrets engine [1] exposes cryptographic operations over an authenticated HTTPS API. Keys are created with a name and a type (`rsa-2048`, `ecdsa-p256`, `ed25519`, `aes256-gcm96`, etc.) and are never exportable by default. The engine supports:

- **Sign / Verify**: `POST /v1/transit/sign/:name` accepts a base64-encoded input (raw data or a digest), an optional hash algorithm, and a signature algorithm (`pss`, `pkcs1v15`, `ecdsa`).

Listing 1: Key reference structure.

```
typedef struct {
    char          *key_name;
    int            key_version;
    char          *key_type;
    unsigned char *pubkey_der;
    size_t         pubkey_der_len;
} vault_keyref_t;
```

- **Encrypt / Decrypt**: symmetric and RSA operations; ciphertext is returned as a versioned envelope string (`vault:vN:base64`), where *N* is the key version used for encryption.
- **HMAC**: `POST /v1/transit/hmac/:name/:alg`.
- **Key metadata**: `GET /v1/transit/keys/:name` returns the key type, version history, and PEM-encoded public keys.

Access is governed by Vault policies; every operation is recorded in the audit log, giving operators a complete and tamper-evident record of key usage.

3 Design

3.1 The Central Invariant: Keys Never Leave Vault

The defining property of openssl-provider-vault is that *no private key material is ever present in the process memory of the application using OpenSSL*. The provider holds only a *key reference* struct (`vault_keyref_t`) containing:

- the Vault key name and current version number,
- the key type string (`"rsa-2048"`, `"ecdsa-p256"`, *etc.*),
- optionally, a cached copy of the *public* key in DER-encoded SubjectPublicKeyInfo format, fetched once at key creation or load time.

The public key is cached locally as a performance optimization for operations that only require it (certificate verification, TLS handshake public-key extraction) and carries no secret material.

When an operation requiring key material is requested—such as signing, decrypting, or HMAC computation—the provider serializes the input according to Vault's API contract, transmits it over an HTTPS connection with standard TLS server authentication, and returns only the result (signature bytes, plaintext, MAC tag) to the caller. The private key is exercised exclusively inside Vault.

3.2 Supported Operations

openssl-provider-vault implements four provider operation classes:

KEYMGMT. Handles key lifecycle: creation (`gen`), loading from a serialized reference (`load`), parameter queries (`get_params`), and public-key export (`export`). Private-key export is unconditionally rejected; the provider returns an error if `OSSL_KEYMGMT_SELECT_PRIVATE_KEY` is requested. Key generation maps to `POST /v1/transit/keys/:name`; the key name is supplied via a custom `OSSL_PARAM` (`"vault-key-name"`).

SIGNATURE. Implements sign, verify, and the digest-sign/verify streaming variants for RSA-PSS, RSA-PKCS#1 v1.5, ECDSA, and Ed25519.

ASYM_CIPHER. Wraps Vault's `encrypt` and `decrypt` endpoints for RSA-based asymmetric encryption.

MAC. Delegates HMAC computation to Vault's `hmac` endpoint.

STORE. Resolves `vault:` URIs to key references. Applications load a Vault-backed key through OpenSSL's STORE framework using a URI such as `"vault:my-signing-key"`, analogous to other URI-backed loaders such as `pkcs11:`.

3.3 Configuration

The provider is activated by adding a stanza to `openssl.cnf`:

Listing 2: Minimal openssl.cnf for openssl-provider-vault.

```
[openssl_init]
providers = provider_sect

[provider_sect]
vault = vault_sect

[vault_sect]
module = /usr/lib/ossl-modules/vault.so
vault_addr  = https://vault.example.com
    :8200
vault_token = s.xxxxx
```

No application code changes are required beyond configuring OpenSSL to load the provider and supply the necessary Vault connection parameters.

4 Implementation

openssl-provider-vault is implemented in approximately 2 600 lines of C99 across ten source modules (Table 1). It depends on libcurl for HTTP transport and json-c for response parsing; all other dependencies are satisfied by the OpenSSL library itself.

4.1 Testing

At the time of writing, the test suite comprises roughly 3 500 lines of test code and just over 150 unit and integration tests across nine binaries. It is designed so that every test binary is entirely self-contained: each links the source files it exercises directly, rather than loading the provider shared library. Where a test exercises a module that normally calls vault_client.c, a *mock vault client* is substituted, allowing full control over Vault API responses via cmocka's `will_return` mechanism [4].

Table 1: Source modules and their roles.

Module	Responsibility
`provider.c`	Provider entry point, context lifecycle
`vault_client.c`	HTTP client (libcurl), JSON parsing
`vault_format.c`	Base64 encode/decode, Vault wire format
`vault_keyref.c`	Key reference struct, serialization
`vault_store_uri.c`	`vault:` URI parser
`keymgmt.c`	KEYMGMT dispatch implementation
`signature.c`	SIGNATURE dispatch implementation
`asym_cipher.c`	ASYM_CIPHER dispatch implementation
`mac.c`	MAC (HMAC) dispatch implementation
`store.c`	STORE dispatch implementation

One test binary (`test_vault_client`) is an integration test that starts a local Vault dev server automatically if the `vault(1)` binary is present on `PATH`, or connects to an external server if `VAULT_ADDR`/`VAULT_TOKEN` are set. If neither condition is met, the binary exits with code 77 (autotest "skip").

OOM / allocation-failure testing. Heap exhaustion is a common cause of subtle memory-safety bugs in C code. To exercise all allocation failure paths, we use GNU ld's `–wrap` mechanism to intercept `malloc` and `calloc` at link time. A shared helper (`wrap_malloc.c`) exposes a `wrap_malloc_fail_after(n)` function that causes the next allocation from compiled-in code to return `NULL` after *n* successful calls. Because the system libraries (libcrypto, libcurl, libc) are shared objects, their internal allocations are *not* intercepted—only direct calls from provider source files are affected, giving precise control over which allocation is under test.

5 Design Challenges

5.1 The Prehashing Impedance Mismatch

OpenSSL's digest-sign API is a streaming interface: the caller feeds data incrementally through `DigestSignUpdate` and finalizes with `DigestSignFinal`. Vault's sign endpoint, by contrast, accepts a single pre-digested input alongside the `prehashed=true` flag.

The provider resolves this by maintaining an `EVP_MD_CTX` inside the signature context struct. During update calls, data is accumulated into the local digest context; at finalization time, the digest is computed locally and the resulting hash

is submitted to Vault. This is correct and secure: the hash is computed in the same process that would otherwise compute it before handing off to any software signer; no trust boundary is crossed.

5.2 Context Duplication (dupctx)

The OpenSSL provider interface requires that a context can be duplicated at any point during a streaming operation via `dupctx`. For the signature provider, this means copying the `EVP_MD_CTX` mid-stream. The implementation calls `EVP_MD_CTX_copy_ex` on the source context and attaches the result to the newly allocated destination context. Care must be taken to null-initialize the destination's `mdctx` pointer before branching on the source, so that a copy-failure path calls `EVP_MD_CTX_free(NULL)` (a no-op) rather than double-freeing a partially initialized struct.

5.3 Vault Wire Format

Vault's transit engine encodes all binary data in URL-safe base64 (no padding in some contexts) and prefixes ciphertext with a versioned envelope (`vault:vN:base64`), where N is the key version number. The `vault_format.c` module centralizes all encoding and decoding using OpenSSL's BIO chain API, ensuring a single, tested code path for all format conversions.

5.4 Algorithm Identifier Generation

Some callers (notably those constructing X.509 certificate signing requests) query the signature context for an ASN.1 `AlgorithmIdentifier` via the `OSSL_SIGNATURE_PARAM_ALGORITHM_ID` parameter. The provider synthesizes the correct DER-encoded structure using `X509_ALGOR_new` and `i2d_X509_ALGOR`, driven by the hash NID and key type stored in the context. This is necessary for compatibility with OpenSSL's CSR and certificate-generation code paths, which retrieve the algorithm identifier from the signer context rather than from the key.

6 Security Properties and Limitations

6.1 What openssl-provider-vault Protects

Memory-resident key material. Because the private key is never retrieved from Vault, a process-level compromise—including a full heap or stack dump, an arbitrary-read vulnerability, or a debugger attach—cannot yield the private key. This is the primary threat addressed.

Key-use audit trail. Every signing, encryption, and HMAC operation generates an entry in Vault's audit log. Operators can detect unexpected key usage, enforce rate limits via Vault policies, and obtain a forensic timeline after an incident.

Key revocation and rotation. Vault supports immediate key revocation and seamless key rotation without re-keying existing ciphertexts (for symmetric keys). The `vault_keyref_t` stores the key version used at operation time, enabling Vault's min-decryption-version policy to enforce forward secrecy windows.

Access control. Vault policies can restrict which application identities (AppRole, Kubernetes auth, etc.) may use which keys for which operations, with much finer granularity than file system permissions.

6.2 Residual Risks

Vault availability. openssl-provider-vault introduces a synchronous dependency on the Vault cluster for every private-key operation. A Vault outage or network partition will prevent signing and decryption. Operators should deploy Vault with its recommended HA configuration [7].

Transport security. The provider communicates with Vault over HTTPS. The security of this channel depends on correct certificate validation and the absence of a MITM adversary. Currently the provider relies on libcurl's default server-certificate validation; future work should expose configuration for a pinned CA bundle and, optionally, client-certificate-based authentication.

Token storage. The Vault token is currently read from environment variables or `openssl.cnf`. An adversary with read access to either can obtain a token and use it to request signatures directly. More robust authentication methods (e.g., Vault Agent, Kubernetes auth, AWS IAM auth) should be preferred in production, and token storage should be treated with the same care as the key it protects.

Side channels in Vault. openssl-provider-vault does not address timing or other side channels within the Vault process itself. Vault uses Go's standard library crypto, which incorporates timing countermeasures—RSA blinding, constant-time comparison via `crypto/subtle`—for critical operations. The extent to which Go's big-number arithmetic provides complete constant-time guarantees remains an area of active research [6]; this dimension of side-channel resistance is outside the scope of the present work.

Scope: no FIPS certification. openssl-provider-vault is not a FIPS 140-validated module. Organizations with FIPS requirements should evaluate whether Vault's own FIPS variant satisfies their compliance needs.

7 Related Work

PKCS#11-based providers. `pkcs11-provider` [3] is the primary prior art for OpenSSL 3 provider-based key

isolation. It implements the same drop-in transparency goal against PKCS#11 tokens—physical HSMs, SoftHSM, TPMs. openssl-provider-vault is complementary: it targets deployments where a cloud-native secret-management system is preferred over a hardware device, and where Vault's policy engine and audit capabilities provide value beyond raw key isolation.

Post-quantum providers. The Open Quantum Safe project's `oqs-provider` [5] demonstrates the same OpenSSL 3 provider dispatch pattern in a different domain: it registers quantum-safe KEM and signature algorithms so that TLS 1.3, X.509, and S/MIME consumers can use them transparently. While its focus is algorithmic diversity rather than key isolation, it is a notable example of the provider model's generality and of the ecosystem of third-party providers that have emerged since OpenSSL 3.0.

Engine-based approaches. OpenSSL's deprecated engine interface was used by numerous projects (e.g., `engine_pkcs11`, `aws-lc`) to redirect private-key operations. The provider model is the supported successor and avoids the thread-safety and context-propagation problems inherent in the global engine registry.

Vault PKI / TLS integration. HashiCorp provides several Vault-adjacent tools for TLS (Vault Agent, Envoy SDS, cert-manager) but these operate at the certificate issuance layer, not at the signing primitive layer. They do not prevent key material from residing in the application process after issuance. openssl-provider-vault addresses the residual risk of an already-issued key being exposed at runtime.

8 Future Work

Performance characterization. A systematic latency and throughput study across key types, Vault deployment topologies (local, Vault Cloud, HA cluster), and TLS session reuse strategies is needed to quantify the performance overhead of the remote-dispatch model.

mTLS and certificate-based Vault authentication. Replacing token authentication with a client-certificate-based Vault auth method would eliminate the token-storage risk identified in §6.

Key derivation context. Vault's `derived: true` key flag enables per-context key derivation. Exposing this through a custom `OSSL_PARAM` would support authenticated encryption schemes where each session derives a unique key from a shared root.

Asynchronous signing. Long-lived TLS servers that sign thousands of handshakes per second must serialize all signing operations through the Vault HTTP endpoint. Exposing an asynchronous signing interface and pipelining requests over a persistent connection pool would substantially reduce latency under load.

Digest provider. While hashing inside Vault adds an unnecessary network round-trip for most use cases, the `transit/hash` endpoint could be exposed as an optional provider-backed DIGEST operation for regulated environments that require all cryptographic operations to be recorded in the Vault audit log.

PKCS#11 compatibility layer. An open-source Vault-backed PKCS#11 module would be a natural complement to the provider design for software ecosystems that expect HSM-style interfaces, particularly for deployments that wish to avoid proprietary middleware in their cryptographic stack. Such a module could expose Vault-managed keys through the de facto standard token API used by existing HSM integrations, while reusing much of the same remote-operation model described in this work.

9 Conclusion

We have presented openssl-provider-vault, an OpenSSL 3 provider that delegates private-key operations to HashiCorp Vault's transit secrets engine. By implementing the standard OpenSSL provider dispatch interface, the system achieves drop-in transparency: any application or library built on the OpenSSL EVP API—from `openssl(1)` to production TLS servers—gains the isolation and auditability of Vault-managed keys without modification. The provider handles the prehashing impedance mismatch between OpenSSL's streaming interface and Vault's prehash-oriented REST API, correctly manages context duplication mid-stream, and generates standard ASN.1 algorithm identifiers required by certificate-issuance code paths.

The design ensures that private key material is never present in application process memory; compromise of the application, its heap, or its TLS library cannot yield the key. We have described the residual risks—chiefly Vault availability, token storage, and transport security—and outlined mitigations for each.

openssl-provider-vault is open-source and available at https://github.com/sorenisanerd/openssl-provider-vault.

References

[1] HashiCorp. Vault Transit Secrets Engine Documentation. https://developer.hashicorp.com/vault/docs/secrets/transit, 2024.

[2] OpenSSL Project. Providers, OpenSSL 3.x Manual. https://www.openssl.org/docs/man3.0/man7/provider.html, 2024.

[3] Simo Sorce et al. pkcs11-provider: A PKCS#11 Provider for OpenSSL 3. https://github.com/openssl-projects/pkcs11-provider, 2023.

[4] Andreas Schneider and Google Inc. cmocka: Unit testing framework for C. https://cmocka.org, 2013.

[5] Open Quantum Safe Project. oqs-provider: An OpenSSL 3 Provider for Quantum-Safe Cryptography. https://github.com/open-quantum-safe/oqs-provider, 2025.

[6] Lucas Meier. Constant-Time Big Numbers (for Go). EPFL Technical Report, 2021. https://www.epfl.ch/labs/dedis/wp-content/uploads/2021/07/report-2021-1-LucasMeier_ConstantTime.pdf

[7] HashiCorp. Vault High Availability Documentation. https://developer.hashicorp.com/vault/docs/concepts/ha, 2024.